AI Transparency and Validation of Scientific Computing Methods

Candidate Number: 1085717

Abstract: This report outlines the importance of validating AI-generated code, proposes a method to evaluate its correctness, quality, and security, and provides an example of validating AI-generated convolutional filters.

Introduction to Generative AI

Generative AI (Artificial Intelligence) creates new content by replicating patterns from datasets. According to the 2025 Stack Overflow Developer Survey,[1] 84% of software developers reported they were using generative AI tools in their workflow, which is an increase of 76% since 2024. This trend indicates that generative AI tools have many benefits, such as increasing productivity by allowing for faster code generation and prototyping. However, AI-generated code can often be poor-quality or incorrect due to AI's lacking contextual awareness. AI-generated code can also introduce vulnerabilities such as SQL Injection, which can result in proprietary code or personal information being leaked[2]. Therefore, AI-generated code must be validated before it is integrated into codebases.

Validating AI-generated Code

The primary criterion proposed in this report for validating AI-generated code is correctness. This is defined as achieving pre-defined specifications regardless of the input, and being free from errors. To identify bugs that cause errors, the static code analyser SonarQube[3] and bytecode analyser pyLeak[4] have been used. These tools have been chosen as they are complementary and target different issues, with SonarQube excelling at macroscopic structural logic errors, whilst pyLeak excels at byte size errors and memory leaks.

The secondary criterion proposed for validating code is quality. High-quality code is defined as being effective, fast, space-efficient, and maintainable. The following report will use 'Code Runtime' as a metric for time complexity, 'Peak Memory Allocation' as a metric for memory complexity, and 'Cyclomatic Complexity' (a metric that measures the number of linearly independent code paths) for readability. AI-generated code should only be integrated if its quality is similar to or superior to human-written code.

The tertiary criterion proposed for validating code is security. AI-generated code should only be integrated into codebases if it contains no weaknesses that attackers can exploit to compromise confidentiality.

Convolutional Denoising Filters

This report uses denoising filters as an example of validating AI-generated code. This was chosen as generating denoising filters encapsulates core capabilities that AI models rely on, such as signal extraction and convolution. There are a multitude of denoising methods, which allow the AI models to use separate approaches, leading to

---

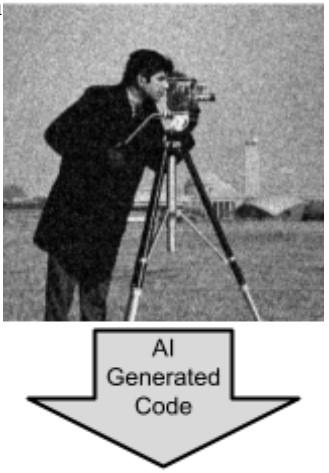
[1] Stack Overflow, 'Developer Survey', https://survey.stackoverflow.co/2025/ai? [Retrieved 21/01/2026]
[2] Schreiber, Maximilian, and Pascal Tippe. *Security Vulnerabilities in AI-Generated Code: A Large-Scale Analysis of Public GitHub Repositories*. Vol. 16219. 2026.
[3] Sonar Source, 'SonarQube', https://www.sonarsource.com/products/sonarqube [Retrieved 21/01/2026]
[4] Github, 'pyLeak', https://github.com/deepankarm/pyleak [Retrieved 21/01/2026]

meaningful distinctions. Denoising filters' performance is also easy to measure and compare using the metric PSNR (Peak Signal to Noise Ratio), which is an approximation of human perception of reconstruction quality[5]. For a monochrome image it is defined as…

$$PSNR = 10 \bullet log_{10}\left(\frac{MAX_I^2}{\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=0}^{n}(I(i,j)-K(i,j))^2}\right)$$

Where: MAX_I is the maximum possible pixel intensity; m is the image's pixel height, n is the image's pixel width, I is the original image's pixel intensities, and K is the investigated noisy image's pixel intensities. The specification for a denoising filter to be correct is that the PSNR of the filtered image needs to always be higher than that of the input image. The report will also use the magnitude of PSNR to determine the effectiveness of a filter.

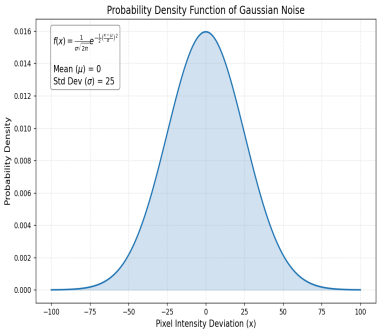
Fig 1. Denoising Filter.



The report uses images from the 'Set12' dataset. This dataset was chosen as it includes a variety of textures, is computationally lightweight, and is a standardised benchmark for denoising.[6]

Methodology

Fig 2. Gaussian Noise.



First, the original image's pixel intensities were measured and converted into 2D arrays of 8-bit values. A histogram was then created by looping through these intensities and incrementing a frequency counter. Next, the noisy images were generated by superposing the original images with Gaussian noise. Values were then clamped from 0 to 255.

Subsequently, four different generative AI models (GPT-4.1, GPT-5.2, Google Gemini 3, and CoPilot) were prompted to write a function to denoise the noisy images. They were provided with three different prompts with increasing detail. The report's author also wrote a filter themselves for comparison.

Fig 3. Prompts

Prompt 1 (Least Detailed):
"Define a Python function to reduce the noise of a grayscale image. Do not import any libraries."

Prompt 2 (Moderately Detailed):
"Define a Python function to reduce the Gaussian noise (sigma = 25) of an 8-bit grayscale image. You should attempt to maximise PSNR whilst minimising time and memory complexity. Do not import any libraries."

Prompt 3 (Most Detailed):
"Define an advanced convolutional filter function to reduce the Gaussian noise (sigma = 25) of an 8-bit grayscale image in Python. Effective nonlinear techniques like Weiner or Patch Based Denoising are recommended. You should attempt to maximise PSNR whilst minimising time and memory complexity. Do not import any libraries."

[5] Sara, Umme, Morium Akter, and Mohammad Shorif Uddin. 'Image Quality Assessment through FSIM, SSIM, MSE and PSNR—A Comparative Study'. *Journal of Computer and Communications* 07, no. 03 (2019): 8–18..
[6] Zhang, Kai, Wangmeng Zuo, Yunjin Chen, Deyu Meng, and Lei Zhang. 'Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising'. *IEEE Transactions on Image Processing* 26, no. 7 (2017): 3142–55. https://doi.org/10.1109/TIP.2017.2662206.

Finally, the PSNR values and histograms of the filtered images were calculated, recorded, and displayed. The ‘Code Runtime’ and ‘Peak Memory Allocation’ were measured using the ‘time’ and ‘tracemalloc’ libraries in Python. ‘Cyclomatic Complexity’, bugs, and vulnerabilities were measured using SonarQube and pyLeak.

Results

Fig 4. Original, Noisy, and Filtered Images. AI-generated Filters are Using Prompt 3.

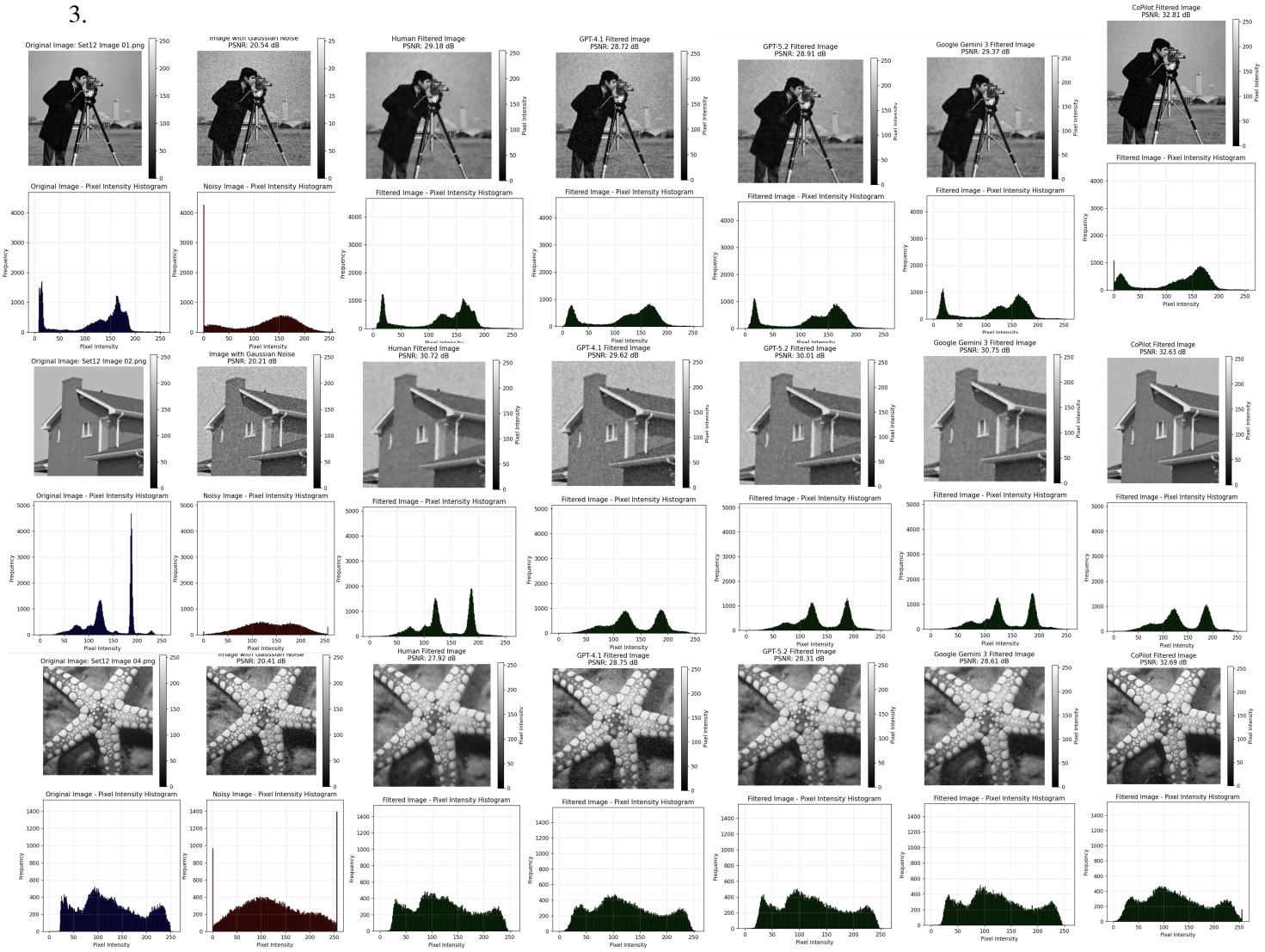


Fig 5. Correctness and Quality Metrics for Validating AI-generated Code.

			Prompt 1 (Least Detailed Prompt)					Prompt 2 (Moderately Detailed Prompt)					Prompt 3 (Most Detailed Prompt)				
Parameters	Noisy Image	Human Code	GPT-4.1	GPT-5.2	Google Gemini	CoPilot	Average	GPT-4.1	GPT-5.2	Google Gemini	CoPilot	Average	GPT-4.1	GPT-5.2	Google Gemini	CoPilot	Average Excluding CoPilot
Filter Type	N/A	Patch-based	Mean	Median	Median	Gaussian	N/A	Weighted-Average	Gaussian	Binomial	Bilateral	N/A	Wiener	Wiener	Wiener	Patch-based	N/A
PSNR - Image 1 (dB)	20.54	29.18	24.43	24.39	24.39	24.77	24.50	26.99	24.77	25.67	24.82	25.56	28.72	28.91	29.37	32.81	29.00
PSNR - Image 2 (dB)	20.21	30.72	27.90	26.69	26.69	28.64	27.48	27.89	28.64	28.73	25.77	27.76	29.62	30.01	30.75	32.63	30.13
PSNR - Image 3 (dB)	20.33	29.14	25.12	23.69	23.69	26.30	24.70	25.12	26.30	26.22	25.69	25.83	29.20	29.10	29.68	32.71	29.33
PSNR - Image 4 (dB)	20.41	27.92	25.76	25.58	25.58	25.73	25.66	25.29	25.73	26.26	24.48	25.44	28.75	28.31	28.61	32.69	28.56
PSNR - Image 5 (dB)	20.26	27.90	26.20	25.49	25.49	26.32	25.88	25.81	26.32	25.63	25.47	25.81	28.63	28.32	28.74	32.70	28.56
Average PSNR (dB)	20.35	**28.97**	25.88	25.17	25.17	26.35	**25.64**	26.22	26.35	26.50	25.25	**26.08**	28.98	28.93	29.43	32.71	**29.11**
SonarQube/ pyLeak Bugs Detected	N/A	0	0	0	0	0	0.00	0	0	0	0	0.00	0	0	1	1	0.33
Code Runtime (s)	N/A	11.44	1.44	1.59	3.57	5.31	2.98	2.00	1.12	0.66	9.49	3.32	5.25	3.85	3.25	61.87	4.12
Peak Memory Allocation (MB)	N/A	20.80	10.40	12.39	10.40	8.86	10.51	7.86	10.40	11.89	7.86	9.50	10.40	10.38	11.48	37.44	10.75

Cyclomatic Complexity	N/A	6	4	4	4	4	4	4	7	10	7	7	8	11	14	14	11
SonarQube/ pyLeak Vulnerabilities	N/A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

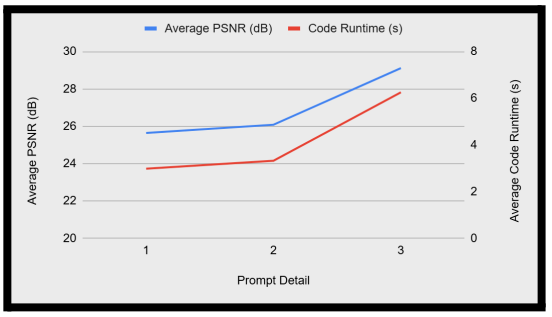
Discussion

All the AI-generated code passed the primary criteria for correctness, as the PSNR values of the filtered images were always greater than those of the noisy image, and the bugs detected by SonarQube did not cause errors (they were caused by assigned variables not being used). The results show that the PSNR values were lowest on average for Image 4 (the starfish). This is because of the extreme intensities being clipped to 0 and 255 after the noise was added, losing information from the signal, as shown in the noisy image histogram in Figure 4.

Excluding the code written by CoPilot for Prompt 3, the AI-generated code also passed the second criterion for quality, as it had lower time and memory complexities than the human-written code, whilst having similar PSNR values and readability. This is likely because the AI adopted more computationally efficient algorithms. For example, the mean filter has a time complexity of $O(mnk^2)$ where k is the kernel size, whilst the patch-based filter has a time complexity of $O(m^2n^2k^2)$ where k is the patch radius. The AI-generated code also used the 8-bit byte datatype, instead of the 32-bit int datatype used in the human code.

Finally, the AI-generated code did not include any vulnerabilities, which was expected due to the code being a local script. Therefore, excluding the code written by CoPilot for prompt 3, the report proposes that the AI-generated code has been validated and can be integrated into a codebase.

Fig 6. Code Effectiveness vs Runtime.



Another trend shown in Figure 6 is that more prompt detail resulted in the AI-generated code being more effective, but resulted in 'Code runtime', 'Peak Memory Allocation', and 'Cyclomatic Complexity' also increasing, making the code slower, more memory-intensive, and harder to read.

Conclusions and Future Work

The following report demonstrates that implementing AI-generated code can be very helpful and fast when it is checked that it is correct, high-quality, and secure. However, to reach the effectiveness of human-written code, the user must have an understanding of the underlying methods, as only once the Generative AI models received the most detailed prompt did they manage to surpass the human code's effectiveness (PSNR). The scope of this report could be expanded in the future by validating AI-generated code of different scientific coding methods.